UR5e ROS 2 Foolproof Startup Guide (ASCII, Full)

All-ASCII, copy-pasteable commands. Long lines are wrapped with shell line-continuation backslashes so they fit on the page and still work in a terminal.

Terminal 1 - Launch the UR driver

```
# Source ROS and your workspace
source /opt/ros/humble/setup.bash
source ~/workspace/ros_ur_driver/install/setup.bash

# Launch the driver (keep this terminal running)
ros2 launch ur_robot_driver ur_control.launch.py \
  ur_type:=ur5e \
  robot_ip:=192.168.57.101 \
  launch_rviz:=false

# Pendant must have External Control loaded (IP=192.168.57.1, Port=50002), robot in Remote.
```

Terminal 2 - Power, brakes, program, speed

```
source /opt/ros/humble/setup.bash
source ~/workspace/ros_ur_driver/install/setup.bash

# Power ON and release brakes
ros2 service call /dashboard_client/power_on std_srvs/srv/Trigger "{}"
ros2 service call /dashboard_client/brake_release std_srvs/srv/Trigger "{}"

# Start External Control program ("play")
ros2 service call /dashboard_client/play std_srvs/srv/Trigger "{}"

# Set speed slider (50%)
# Depending on your UR driver version, the service may be under either of these:
#   ur_msgs/srv/SetSpeedSliderFraction  -->  /io_and_status_controller/set_speed_slider
#   ur_dashboard_msgs/srv/SetSpeedSliderFraction  -->  /dashboard_client/set_speed_slider
# Try the first one first (most Humble setups use ur_msgs):
ros2 service call /io_and_status_controller/set_speed_slider \
  ur_msgs/srv/SetSpeedSliderFraction "{speed_slider_fraction: 0.5}"

# Confirm program is running and safety mode is normal
ros2 topic echo --once /io_and_status_controller/robot_program_running
ros2 topic echo --once /io_and_status_controller/safety_mode
```

Terminal 3 - Ensure scaled_joint_trajectory_controller is active

```
source /opt/ros/humble/setup.bash
source ~/workspace/ros_ur_driver/install/setup.bash

# Check current controllers
ros2 control list_controllers

# Switch controllers (Humble-compatible service call; uses Duration for timeout)
ros2 service call /controller_manager/switch_controller \
  controller_manager_msgs/srv/SwitchController "{ \
    start_controllers: ['scaled_joint_trajectory_controller'], \
    stop_controllers: ['joint_trajectory_controller', 'passthrough_trajectory_controller'], \
    \
    strictness: 2, \
    activate_asap: true, \
    timeout: {sec: 5, nanosec: 0} \
  }"

# If you need to load first (only once per boot/session):
ros2 control load_controller scaled_joint_trajectory_controller
```

```
# Notes:
# - 'Error loading controller' when it's already loaded is normal.
# - 'cannot activate ... from its current state active' means it's already active (also \
normal).

# Verify active
ros2 control list_controllers | grep scaled_joint_trajectory_controller
```

Controller management - activate/deactivate/unload (reference)

```
# List current controllers and states
ros2 control list_controllers

# Activate a controller (if already loaded/configured)
ros2 control set_controller_state scaled_joint_trajectory_controller active

# Deactivate a controller (keep it loaded but not commanding joints)
ros2 control set_controller_state scaled_joint_trajectory_controller inactive

# Preferred switch (Humble service; avoids joint conflicts)
ros2 service call /controller_manager/switch_controller \
  controller_manager_msgs/srv/SwitchController "{ \
    start_controllers: ['scaled_joint_trajectory_controller'], \
    stop_controllers: ['joint_trajectory_controller', 'passthrough_trajectory_controller'], \
    \
    strictness: 2, \
    activate_asap: true, \
    timeout: {sec: 5, nanosec: 0} \
  }"

# Unload a controller (removes it until re-loaded)
ros2 control unload_controller joint_trajectory_controller

# Reminder: Broadcasters and status controllers (joint_state_broadcaster, \
speed_scaling_state_broadcaster,
# io_and_status_controller, ur_configuration_controller, tcp_pose_broadcaster, \
force_torque_sensor_broadcaster)
# can remain active; they do not claim joints.
```

Terminal 4 - Send a known-good goal

```
Option A - Safe nudge (0.1 rad on wrist_3)
ros2 action send_goal /scaled_joint_trajectory_controller/follow_joint_trajectory \
  control_msgs/action/FollowJointTrajectory "{ \
    trajectory: { \
      joint_names: [ \
        'shoulder_pan_joint','shoulder_lift_joint','elbow_joint', \
        'wrist_1_joint','wrist_2_joint','wrist_3_joint' \
      ], \
      points: [{ \
        positions: [0.0, 0.0, 0.0, 0.0, 0.0, 0.10], \
        time_from_start: {sec: 2, nanosec: 0} \
      }] \
    } \
  }"

Option B - Ready pose [0, -pi/2, +pi/2, 0, +pi/2, 0] in 2 seconds
ros2 action send_goal /scaled_joint_trajectory_controller/follow_joint_trajectory \
  control_msgs/action/FollowJointTrajectory "{ \
    trajectory: { \
      joint_names: [ \
        'shoulder_pan_joint','shoulder_lift_joint','elbow_joint', \
        'wrist_1_joint','wrist_2_joint','wrist_3_joint' \
      ], \
      points: [{ \
        positions: [0.0, -1.5708, 1.5708, 0.0, 1.5708, 0.0], \
        time_from_start: {sec: 2, nanosec: 0} \
```

```
      }] \
    } \
  }"
```

If robot does not move

```
# Diagnostics
ros2 topic echo --once /io_and_status_controller/robot_program_running
ros2 topic echo --once /io_and_status_controller/robot_mode

# Clear protective stop
ros2 service call /dashboard_client/unlock_protective_stop std_srvs/srv/Trigger "{}"

Then re-run:
  ros2 service call /dashboard_client/play std_srvs/srv/Trigger "{}"
  (activate controller again if needed)
  resend motion goal

This guarantees driver, power, safety, and controller states are synchronized before motion.
```